System Monitor Project Explanation

Generated from explain.txt

System Monitor Project Explanation

Project Overview

This project is a real-time hardware monitor web app. It shows live information about the computer's CPU, RAM, disk, temperature, GPU, and network activity inside a browser dashboard. The app is built with Flask in Python, a Bash monitoring script, and a frontend using HTML, CSS, and JavaScript.

The main goal is to continuously collect system information from macOS and display it in a clean visual interface that updates automatically.

How the Project Is Structured

The project has three layers:

1. Frontend
   - HTML builds the page layout.
   - CSS styles the dashboard.
   - JavaScript fetches new data from the backend every few seconds and updates the screen.

2. Backend
   - Flask in app.py exposes an API endpoint at /api/stats.
   - It runs the monitor.sh script.
   - It parses the output and converts it into JSON.
   - It caches results for a short time to reduce system load.

3. System Data Collection
   - monitor.sh uses macOS system tools like ps, sysctl, vm_stat, df, netstat, powermetrics, smartctl, nvidia-smi, and rocm-smi.
   - It collects actual hardware data and prints it in both human-readable and machine-readable forms.

Files in the Project

1. app.py
   This is the Flask backend.

2. monitor.sh
   This is the Bash script that gathers hardware metrics.

3. templates/index.html
   This is the page structure for the dashboard.

4. static/style.css
   This file controls the visual design.

5. static/script.js
   This file handles live updating of the dashboard.

Detailed Explanation of app.py

Purpose

app.py is the heart of the backend. It receives requests from the browser, decides whether to use cached data or refresh it, runs the monitoring script, parses the results, and sends JSON

back to the frontend.

Important Imports

from flask import Flask, jsonify
- Flask creates the web server.
- jsonify converts Python dictionaries into JSON responses.

import subprocess
- This lets Python run the Bash script monitor.sh.

import re
- This is used to extract specific values from the script output with regular expressions.

from time import time
- This gives the current time in seconds so the cache can be checked.

Cache Variables

```
_stats_cache = {"data": None, "timestamp": 0}
CACHE_TTL = 3
```

- _stats_cache stores the last API response.
- data holds the actual JSON-like dictionary.
- timestamp remembers when the cache was created.
- CACHE_TTL is 3 seconds, meaning the backend will reuse the same result for 3 seconds before running the script again.

Why caching is used:
- monitor.sh runs several commands.
- Those commands are not free; they take time and use system resources.
- Caching makes the dashboard faster and smoother.

API Route

```
@app.route('/api/stats')
def get_stats():
```

This defines the endpoint the frontend calls. Every time the browser fetches /api/stats, this function runs.

Cache Check Logic

```
now = time()
if _stats_cache["data"] and (now - _stats_cache["timestamp"]) < CACHE_TTL:
    return jsonify(_stats_cache["data"])
```

This means:
- Get the current time.
- Check whether data already exists in the cache.
- If the cached data is younger than 3 seconds, send it immediately.
- If not, continue and collect fresh data.

Running the Bash Script

```
raw_output = subprocess.check_output(['bash', 'monitor.sh'], timeout=10,
stderr=subprocess.STDOUT)
raw_output = raw_output.decode('utf-8')
```

- subprocess.check_output runs the monitoring script.
- bash monitor.sh executes the script with Bash.

- timeout=10 prevents the app from hanging forever.
- stderr=subprocess.STDOUT merges error output into normal output.
- decode('utf-8') converts bytes into a normal string.

If an error happens, Flask returns a 500 response with the error message.

How Parsing Works

The Bash script prints lines like:

DATA_CPU: 15.6545

DATA_RAM: 19.00% | Used: 3.58GB | Total: 18.00GB
DATA_TEMP: 45.0

DATA_DISK: 22% | Total: 460Gi | Free: 50Gi | SMART: PASSED
DATA_GPU_VENDOR: Apple
DATA_GPU_TEMP: 45.0

DATA_GPU_FULL: Apple M3 Pro | Usage: 0% | Health: Optimal

Python uses regex to extract the values.

Example:

cpu = re.search(r'DATA_CPU:\s+([0-9.]+)', raw_output)

- DATA_CPU: matches the label.
- \s+ means one or more spaces.
- ([0-9.]+) captures the number.

RAM parsing is more complex because one line contains multiple values.

ram_line = re.search(r'DATA_RAM:\s+(.*)', raw_output)

Then the code extracts:
- percent using ([0-9.]+)%
- used memory using Used:\s*([0-9.]+GB)
- total memory using Total:\s*([0-9.]+GB)

Disk parsing works the same way.

GPU parsing gets:
- vendor
- temperature
- full GPU details

Final JSON Response

The backend sends JSON such as:


```
{
  "cpu": 15.6,
  "ram": 19.0,
  "ram_used": "3.58GB",
  "ram_total": "18.00GB",
  "temp": 45.0,
  "disk": 22.0,
  "disk_total": "460Gi",
  "disk_free": "50Gi",
  "smart": "PASSED",
  "network": "In: ... | Out: ...",
  "gpu_vendor": "Apple",
```

```
  "gpu_temp": "45.0",
  "gpu_full": "Apple M3 Pro | Usage: 0% | Health: Optimal",
  "raw": "full raw output from monitor.sh"
}
```


Detailed Explanation of monitor.sh

Purpose

monitor.sh is the system data collector. It gathers actual hardware data from the operating system and prints it in a format that app.py can parse.

Why Bash is used:

- Bash can call many macOS utilities directly.
- It is good for system-level monitoring.
- It keeps the actual hardware probing outside the Python app.

get_cpu_usage()

This function computes CPU usage.

It uses:

- sysctl -n hw.ncpu to get the number of CPU cores.
- ps -A -o %cpu to list CPU usage of all processes.
- awk to sum the values and divide by the number of cores.

This gives an average CPU usage value.

get_cpu_temp()

This function gets the CPU temperature.

It uses powermetrics, which is a macOS power and thermal utility.
It tries to read the CPU die temperature.
If that is unavailable, it falls back to 45.0.

get_gpu_vendor()

This function detects the GPU vendor.

It checks:

- nvidia-smi for NVIDIA
- rocm-smi for AMD
- otherwise Apple

This allows the script to adapt to different machine types.

get_gpu_temp()

This function gets GPU temperature.

For NVIDIA:

- It uses nvidia-smi --query-gpu=temperature.gpu.

For AMD:

- It uses rocm-smi --showtemp.

For Apple Silicon:

- It returns the CPU die temperature because the GPU and CPU share the thermal package.

This is why on an Apple M3 Pro the GPU temperature matches the CPU temperature.

get_gpu_data()

This function returns a detailed GPU status string.

For NVIDIA:
- temperature
- utilization
- memory usage

For AMD:
- utilization
- health status

For Apple:
- GPU active residency from powermetrics
- health status

This function gives the frontend one human-readable string for display.

get_disk_info()

This function gathers disk usage.

It uses:
- df -h / to get disk percentage, total size, and free space.
- smartctl if available to check SMART health.

The output includes:
- usage percentage
- total size
- free size
- SMART status

get_ram_info()

This function computes RAM usage.

It uses vm_stat, which gives memory statistics in pages.
It collects:
- active pages
- free pages
- speculative pages
- page size

Then it converts pages into gigabytes using bc.

It outputs:
- percentage used
- used GB
- total GB

get_net_info()

This function reads network packet statistics.

It uses netstat -ib and extracts packet counts for en0.

The output looks like:
In: X pkts | Out: Y pkts

The Final Script Output

monitor.sh prints two parts:

1. Human-readable section
    - For debugging in the terminal.
    - Helps the developer see the values clearly.

2. DATA_* section
    - For the Python parser.
    - Includes exact labels like DATA_CPU and DATA_RAM.

This two-part design is useful because it helps both humans and the program.

Detailed Explanation of index.html

Purpose

This file builds the dashboard structure.
It contains placeholders for CPU, RAM, temperature, disk, GPU, and network metrics.

Main Parts

Header
- Shows the title Hardware Dashboard.
- Displays the CPU and GPU model at the top.

System Load Card
- Shows CPU usage.
- Shows RAM usage.
- Includes progress bars.

Thermal Monitor Card
- Shows temperature in degrees Celsius.
- Has a vertical thermometer visual.

Storage & SMART Card
- Shows disk usage.
- Shows SMART health.
- Shows free and total disk space.

GPU Monitor Card
- Shows GPU vendor.
- Shows GPU temperature.
- Shows detailed GPU text.

Network I/O Card
- Shows packet in/out data.

Live Console Report
- Shows the raw monitor.sh output in a preformatted block.

Important IDs

The JavaScript uses IDs to update the page.
Examples:
- cpu-fill
- cpu-txt
- ram-fill
- ram-txt
- temp-txt
- temp-fill
- disk-val

- smart-status
- disk-detail
- gpu-vendor
- gpu-temp-txt
- gpu-detail
- net-stats
- raw-log

Detailed Explanation of style.css

Purpose

style.css makes the dashboard look modern and polished.
It uses a glass-morphism style with a dark background and translucent cards.

Main Design Ideas

1. Dark gradient background
    - Gives the app a modern look.

2. Glass effect
    - Uses background transparency and backdrop blur.

3. Rounded cards
    - Makes the UI softer and cleaner.

4. Smooth transitions
    - Bars and thermometer animate nicely.

5. Responsive grid
    - Cards are arranged in a 2-column layout.

Important CSS Classes

.glass
- Used on cards.
- Adds blur and transparency.

.grid
- Makes the dashboard grid layout.

.card
- Generic container for each monitor panel.

.bar-background and bar-fill
- Used for CPU and RAM bars.
- bar-fill width changes dynamically.

.thermometer and temp-fill
- Used for the temperature display.
- temp-fill height changes dynamically.

.tag
- Used for GPU vendor label.
- Styled like a pill or badge.

.stat-detail
- Small gray text under the main values.

Detailed Explanation of script.js

Purpose

script.js fetches new data from the Flask API and updates the dashboard every 5 seconds.
It is responsible for the live feel of the page.

updateDashboard()

This is the main function.
It does the following:
- fetches /api/stats
- converts the response to JSON
- updates the UI elements with new values

Updating CPU and RAM

The script sets the width of the bars to the percentage values.
It also sets the text values shown next to them.

Updating Temperature

The script maps temperature to a thermometer height.
It uses a minimum temperature of 20°C and a maximum of 100°C.

Formula:

((temp - 20) / (100 - 20)) * 100

Then it clamps the result to the range 0 to 100.
This prevents the thermometer from going below 0% or above 100%.

Temperature Color Gradient

The helper function tempColorForPercent() converts the temperature percentage into a color.

The color flow is:
- green at low temperature
- yellow in the middle
- red at high temperature

This gives the user a quick visual sense of system heat.

Updating GPU Data

The script:
- reads gpu_vendor
- colors the vendor tag
- reads gpu_temp
- displays gpu_full in the detail area

Vendor colors:
- NVIDIA: green
- AMD: red
- Apple: gray

Polling Interval

The script runs:
- once immediately when the page loads
- then every 5 seconds using setInterval(updateDashboard, 5000)

How the Whole Project Works Together

1. User opens the web page.
2. Browser loads HTML, CSS, and JavaScript.
3. JavaScript calls /api/stats.
4. Flask checks the cache.
5. If the cache is fresh, Flask returns cached data.
6. If the cache expired, Flask runs monitor.sh.
7. monitor.sh collects hardware data.
8. Flask parses the output and returns JSON.
9. JavaScript updates the dashboard.
10. The process repeats every 5 seconds.

Why This Design Was Chosen

- Bash is good for system commands.
- Python Flask is good for API logic and parsing.
- JavaScript is good for live browser updates.
- Caching improves performance.
- Regex makes parsing flexible.
- CSS animations make the interface easier to read.

Important Presentation Points

If someone asks why the project uses three languages:
- Bash collects system-level data.
- Python processes and serves that data.
- JavaScript displays it live in the browser.

If someone asks why the GPU temperature shows 45°C on Apple:
- Apple Silicon does not expose a separate GPU temperature easily.
- GPU and CPU share the same thermal package.
- So the app uses CPU die temperature as the GPU temperature fallback.

If someone asks why the app uses caching:
- To avoid running the monitoring script too often.
- To keep the dashboard responsive.
- To reduce system resource usage.

If someone asks how the thermometer animation works:
- Temperature is converted into a percentage.
- The percentage becomes the thermometer height.
- A color gradient changes from green to red.

If someone asks how RAM is computed:
- The app reads page counts from vm_stat.
- It multiplies by page size.
- Then it converts the result into gigabytes.

If someone asks how disk SMART works:
- smartctl checks disk health.
- If available, the script shows PASSED or N/A.

Final Summary

This project is a complete real-time system monitoring dashboard. It combines system programming, backend development, frontend design, caching, parsing, and live UI updates. It is a good example of how different technologies can work together to build a practical monitoring tool.

The most important ideas to remember are:
- monitor.sh gets the data.

- app.py parses and serves the data.
- script.js displays the data.
- style.css makes it look good.
- caching keeps it fast.

That is the full explanation of how the project works.